BCJRFormer ASIC Architecture

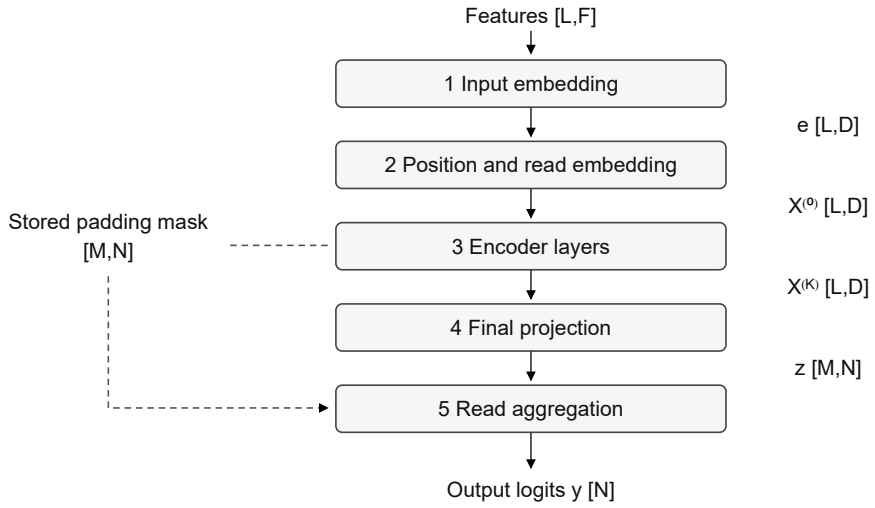
Design Review

Taimor Sabek and Haiel Bader

1 Scope and architecture

We propose a sequential ASIC implementation of the BCJRFormer inference core, from prepared window features and a padding mask to decoded-position logits. The design reuses one dot-product engine and one encoder-layer engine. This reduces arithmetic replication at the cost of throughput. Trellis preprocessing, training and outer decoding are outside the core.

One frame is active at a time ($B = 1$); host batches run sequentially. Let M be the number of read slots, N the positions per read, $L = MN$ the token count, F the flattened window width, D the embedding width, H the head count and K the layer count. Head width is $d = D/H$. Dimensions are fixed from a configuration matching the checkpoint, with D divisible by H.



Each arrow represents a tensor stored in memory. Stages execute in order; the next stage starts only after the previous stage's output writes have committed. The mask is retained for the whole frame. A separate parameter memory supplies learned weights, biases, embedding tables and normalization parameters.

Frame control. Load features and mask, run stages 1–5, transfer the N logits, then report completion. Parameters are loaded and committed before accepting a frame and remain available across frames. Output backpressure cannot discard or duplicate results.

2 Embedding and encoder layer

Stage 1 Input embedding

Input $x_t \in \mathbb{R}^F$ is mapped to $e_t \in \mathbb{R}^D$:

$$u_t = \mathrm{LN}_F(x_t), \qquad v_t = W^{emb} u_t + b^{emb}, \qquad e_t = \mathrm{LN}_D(v_t).$$

Complete after all L tokens. The two normalizations have separate learned parameters. Projection input and output use separate scratch storage because every output needs the complete input vector.

Stage 2 Position and read embedding

For read m, position n and token $t = mN + n$:

$$a_t = e_t + P_n, \qquad X_t^{(0)} = a_t + E_m.$$

$P[N, D]$ and $E[M, D]$ are learned tables loaded from the checkpoint. Inference does not generate them.

Exact integer counters advance feature, position, then read; no runtime modulo is required. Writes update X in place. Complete after LD features; the mask bypasses this stage.

Stage 3 Reused encoder layer

One physical layer executes K times with distinct parameter sets. For each layer:

$$U = \mathrm{LN}_1(X), \quad O = \mathrm{Attention}(U), \quad R = X + O,$$

$$T = \mathrm{LN}_2(R), \quad G = \mathrm{GELU}(TW_1^{\mathsf{T}} + b_1)W_2^{\mathsf{T}} + b_2, \quad X_{next} = R + G.$$

The FFN expands $D \to 4D$ and projects $4D \to D$. Dropout is disabled during inference.

The encoder then advances the parameter bases or releases the final $X^{(K)}[L, D]$ to Stage 4. Original X survives until residual 1; R survives until residual 2. This preserves both bypass connections.

Normalization and FFN control

For vector length $J \in \{F, D\}$:

$$\mu = \frac{1}{J}\sum_j x_j, \quad v = \frac{1}{J}\sum_j (x_j - \mu)^2, \quad y_j = \gamma_j \frac{x_j - \mu}{\sqrt{v + \epsilon}} + \beta_j, \qquad \epsilon = 10^{-5}.$$

The normalization FSM sums inputs, computes the mean, accumulates centered squared differences, computes inverse square root, then normalizes and writes each feature. Statistics are recomputed per token; gamma and beta are learned. Wider intermediate formats must preserve epsilon.

The FFN controller completes linear 1, applies GELU to all $4D$ hidden values, then runs linear 2. The hidden vector remains stored until all D outputs complete. Residual controllers capture both operands before an in-place add/convert/write.

3 Sequential self attention

Projections and scores

Compute all Q tokens, then all K tokens, then all V tokens using the shared linear engine:

$$Q = UW_Q^{\mathsf{T}} + b_Q, \qquad K_{key} = UW_K^{\mathsf{T}} + b_K, \qquad V = UW_V^{\mathsf{T}} + b_V.$$

Here K_{key} denotes the key tensor, distinct from the layer count K. Every projection uses all D input features. Head splitting selects address slices; it does not copy data.

For head a and query i, process one score row over keys j:

$$s_j = \frac{1}{\sqrt{d}} \sum_{r=0}^{d-1} Q_{i,ad+r} (K_{key})_{j,ad+r}.$$

Complete after L scores. The dot uses length d and contiguous operands.

Softmax and masking

$$m_s = \max_j s_j, \qquad e_j = \exp(s_j - m_s), \qquad S = \sum_j e_j, \qquad p_j = e_j / S.$$

Initialize the maximum from the first score and retain the exponential row to avoid recomputation.

The source model masks *query rows*, not key columns. A padded query receives a uniform row $p_j = 1/L$; valid queries can still attend to padded keys. The hardware preserves this behavior using a uniform-value bypass and the same probability writer. Changing mask semantics requires a separate model decision. Optional static masks are excluded from the proposed baseline.

Weighted values and output projection

$$Z_{i,ad+r} = \sum_{j=0}^{L-1} p_j V_{j,ad+r}, \qquad O = ZW_O^{\mathsf{T}} + b_O.$$

Each dot has length L, probability stride 1 and V stride D. Preserve the probability row for all d outputs. Write directly into the joined-head Z bank; no concatenation copy is needed.

Output projection uses the same $D \to D$ linear controller. The layer then adds O to the saved original X.

All Q/K/V tokens are stored, but score, exponential and probability storage is only one row each. Reusing the dot engine for projections, QK and weighted V avoids separate arithmetic engines. The cost is serial execution and additional memory traffic.

4 Output stages and implementation contract

Stage 4 Final projection

Each encoder token becomes one raw logit:

$$z_t = (w^{final})^{\mathsf{T}} X_t^{(K)} + b^{final}.$$

The linear controller owns conversion and writing; no second logit-store state is required. Stage 5 receives all L stored logits.

Stage 5 Read aggregation

For $M > 1$, let $p_{m,n}^{mask} = 1$ indicate padding. Define $c_m = 1$ only when every position of read m is unpadded. The source operation is

$$C = \sum_m c_m, \qquad y_n = \frac{1}{C} \sum_m (1 - p_{m,n}^{mask}) z_{mN+n}.$$

For $M = 1$, copy logits directly without applying the mask. For $C = 0$ in the averaging branch, report an error. Partially padded reads can contribute to the numerator without contributing to C; the accepted padding policy needs confirmation.

The output is N logits. Hard decisions, if needed downstream, use $y_n > 0$; sigmoid is unnecessary for this threshold. Successful frame completion follows the last accepted host output transfer.

Shared resources and control

- **Dot engine:** initialize bias or zero; read A; read B; issue/wait multiply–accumulate; advance term; return a widened sum. Linear control repeats this for output features and owns rounding, saturation and writes.
- **Storage:** parameter memory; X/residual, U, Q/K/V, Z and attention-output banks; token and $4D$ FFN scratch; three length-L row buffers; mask and logit buffers. These are logical banks; SRAM mapping and capacity require the selected dimensions and technology.
- **Transactions:** commands and responses use ready/valid with one outstanding operation per engine. Hold payloads stable during stalls. Advance counters only on accepted work; report completion only after committed writes. Errors stop new work, drain accepted transactions and require reset before retry.

Numerical validation and review decisions

GELU, exponential, reciprocal and inverse square root retain defined request/response interfaces; their implementations and fixed-point formats remain to be selected. Use widened reductions and explicit conversion points. Compare floating-point and bit-accurate intermediate results, record saturation/overflow, then require RTL to match the bit-accurate model. Acceptance must include decoding error rates, not only local arithmetic error.

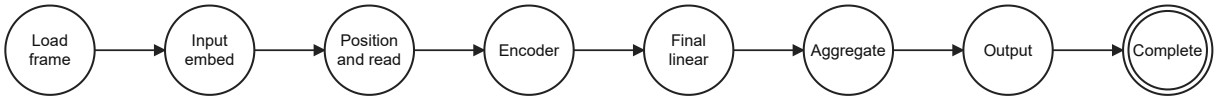
Requested review: confirm the sequential architecture and model/checkpoint; query masking and padding policy; target memory/clock/throughput constraints; and numerical acceptance criteria. These decisions precede arithmetic-format freeze and integrated RTL implementation.

Software basis: BCJRFormer, bcjrformer.py and modules.py, repository revision c7cf84e218730f4583e79af6dd876b04384d33f5.

5 Frame and input control

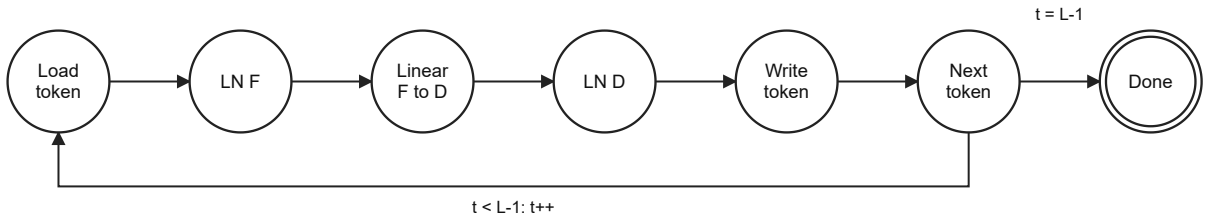
Circles denote states; double circles denote held completion. Enter the first state on accepted command. Unlabeled forward arrows require the current operation to finish. A blocked request, response or write remains in its state until accepted. Read and block-operation states contain request/wait substates. All controllers hold DONE until response acceptance, then return to idle. Fatal errors drain accepted work and report error; reset is required before retry.

Core frame controller



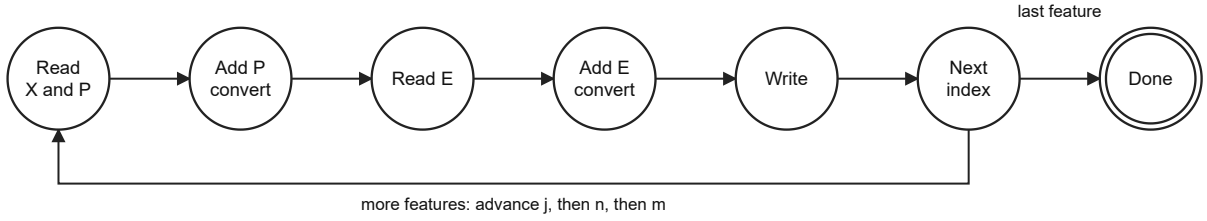
Enter after a frame command is accepted with parameters committed. Output waits for all N host transfers. Complete holds status until accepted; the core then returns to idle.

Stage 1 input embedding



Initialize t=0 on command acceptance. Each child operation waits for completion; Write waits for all D committed features. Normalizations have separate learned parameters.

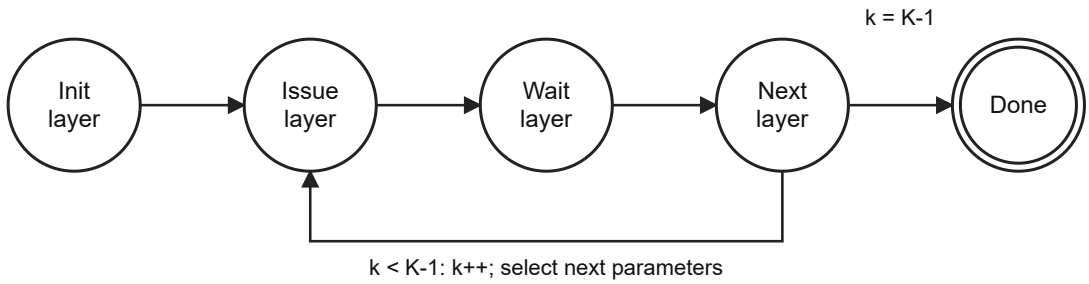
Stage 2 position and read embedding



Initialize m=n=j=0. Read states include request/response waits. Advance exact integer counters only after the write commits; no modulo or quantized indexing.

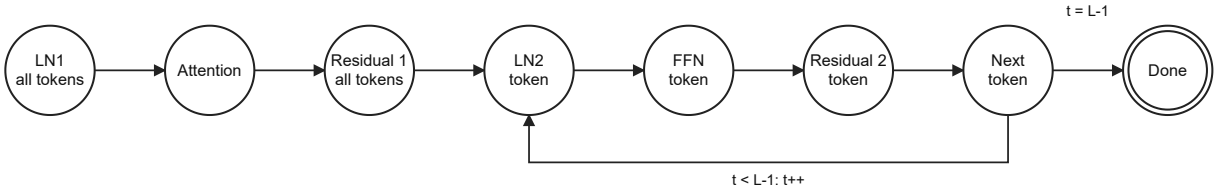
6 Encoder and normalization control

Stage 3 encoder layer loop



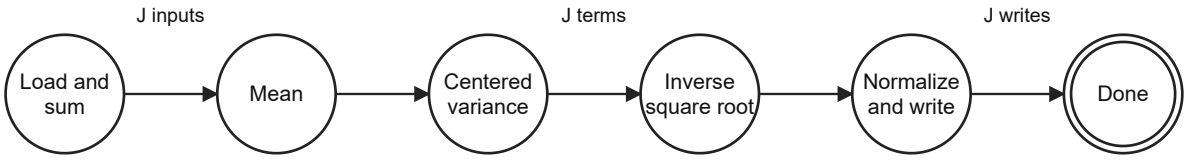
Initialize k=0. Wait layer means the last residual-2 output has committed. One physical layer is reused with distinct learned parameters.

Encoder layer controller



Set t=0 after residual 1. Preserve original X until residual 1 and preserve R until residual 2. Each block state launches once and waits for completion.

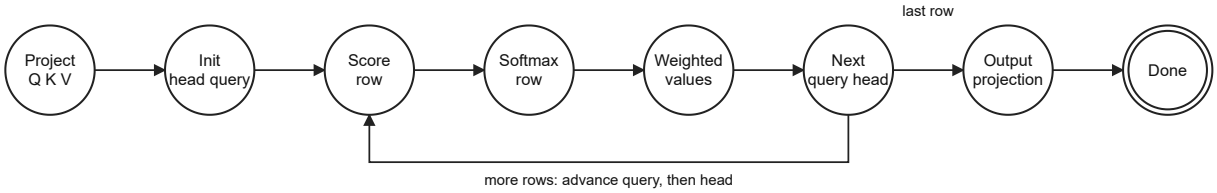
Shared normalization controller



Vector length J is F or D. Reset the index for each pass. Sum, variance and write states internally iterate over J features; inverse square root waits for the scalar response.

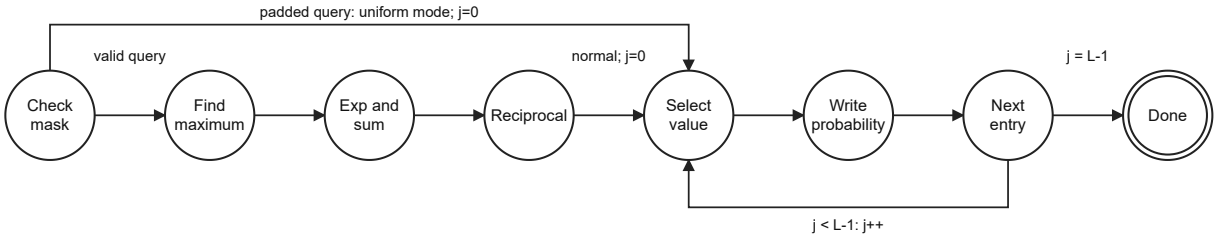
7 Attention and FFN control

Attention head and query controller



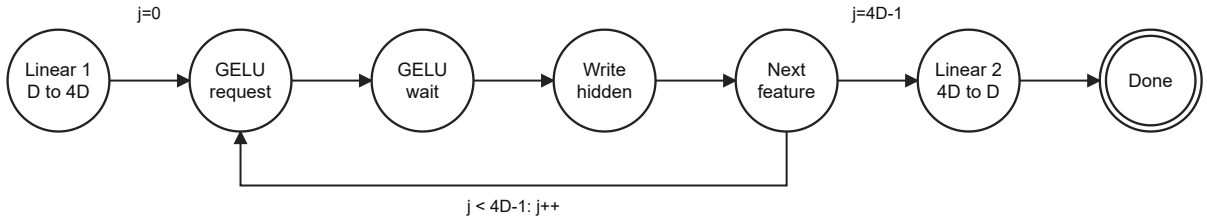
Project all Q tokens, then all K, then all V. Initialize a=i=0. Finish all d weighted outputs before advancing query i; after i=L-1 reset i and advance head a.

Softmax with uniform row bypass



Select value uses stored 1/L in uniform mode, or exp[j] times the saved reciprocal in normal mode. The same writer serves both branches. Nonpositive sum takes the error path.

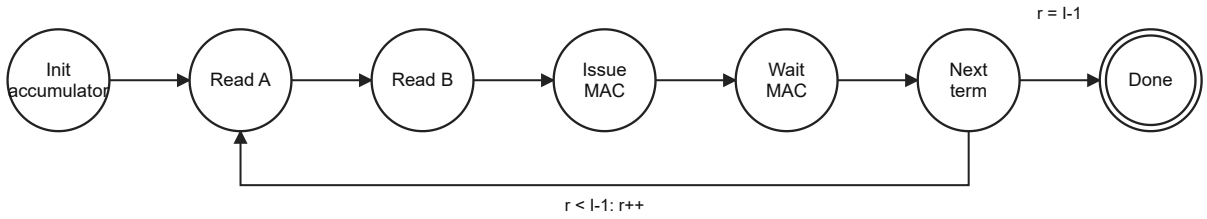
FFN and residual control



Linear states include child-command completion. Preserve the full hidden vector for linear 2. Residual addition is a separate read-left/read-right, widened-add, convert/write loop; both reads precede an in-place write.

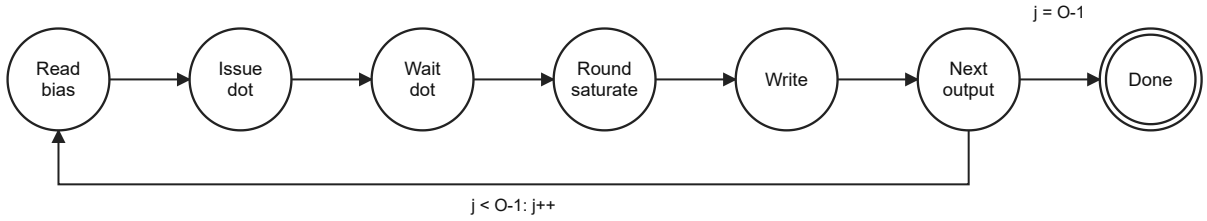
8 Shared arithmetic control

Shared dot product engine



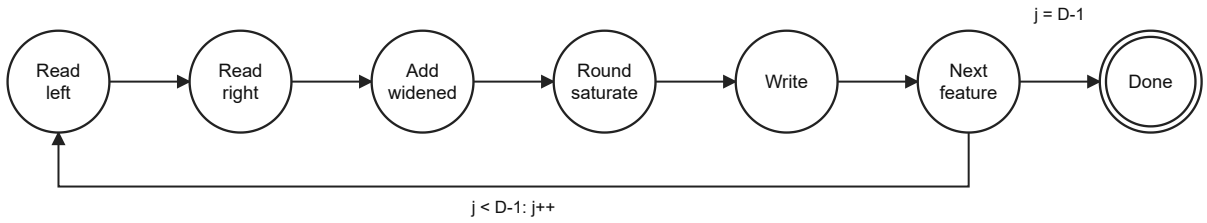
Set r=0 and initialize to aligned bias or zero. Each read includes request/response waits. Issue waits for acceptance; Wait MAC captures one result. Done holds the widened result until accepted.

Shared linear controller



Initialize output j=0 on command acceptance. Dot length is I. Write includes request and committed-write response; only then advance the output index. Done holds completion until accepted.

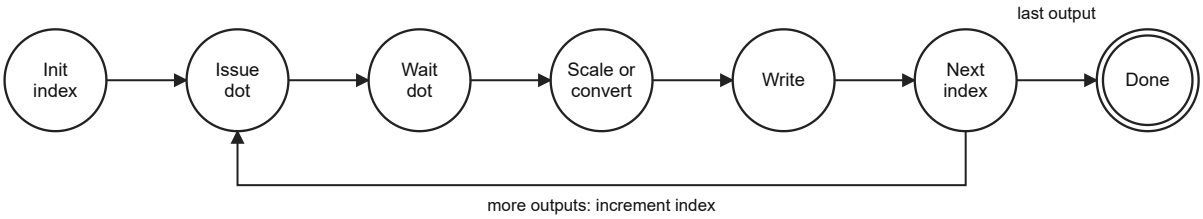
Shared residual addition controller



Initialize j=0. Capture both source operands before an in-place write. Align operand scales before addition. Residual 1 loops over all tokens; residual 2 completes one token at a time.

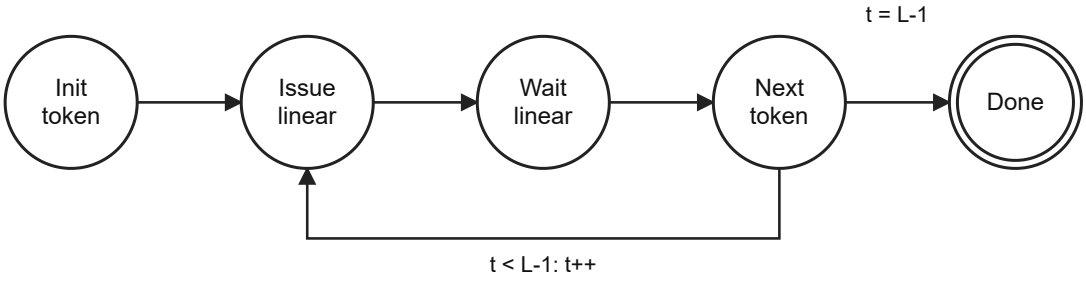
9 Output control

Score row and weighted value controllers



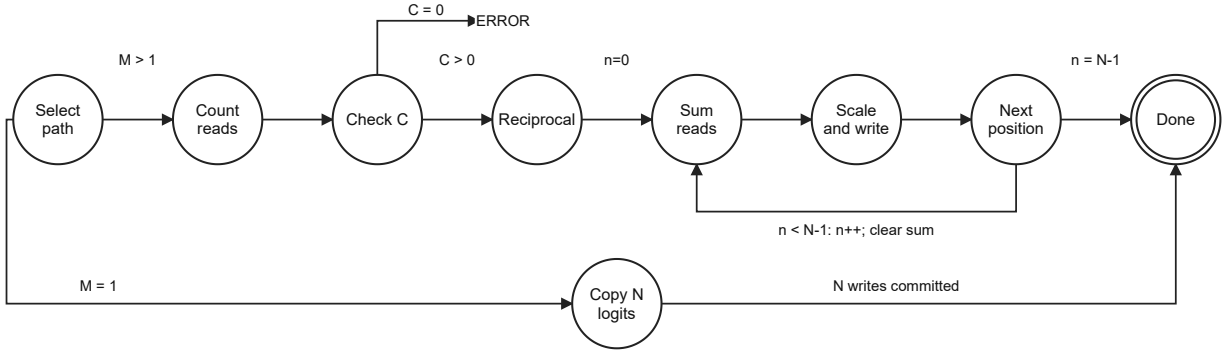
Scores: loop over L keys, use d-term dots, then multiply by stored 1/sqrt(d) and convert. Weighted values: loop over d features, use L-term dots, and convert without extra scaling. Scale and Write include request/completion waits.

Stage 4 final projection and attention output projection



Set t=0. Stage 4 uses D-to-1; attention output projection uses D-to-D. Linear owns conversion and destination writes, so Wait linear includes their commitment.

Stage 5 read aggregation



Compute 1/C once per frame. Copy mode ignores the mask. Averaging uses the source numerator and fully-unpadded-read count. Done means N stored outputs, before host transfer.